

From advertisement text to model features

Saudi real-estate listings — how the title and content columns were turned into structured data

real_estate.parquet — 810,888 listings

title (short headline) content (free-text advertisement body)
87% of the file is these two text columns: content 196 MB, title 22 MB

A. Keyword feature pass — reads title + content

notebook/Exploring_Data_and_sketchs.ipynb

A1 Join the two columns

text = title + " " + content, nulls treated as empty

A2 Normalise the Arabic

emoji and invisible RTL marks to spaces; tatweel and diacritics removed;
alef, ya, ta-marbuta, hamza variants unified; letters repeated 3+ times collapsed

A3 Fold digits

Arabic-Indic digits rewritten as Latin 0-9

A4 Delete negated phrases

"no X" / "without X" / "not X" plus the following words are cut out,
so a denied feature can never match as if it were present

A5 Match 19 keyword patterns

one int8 flag per pattern: ac, parking, pool, kitchen, driver room, basement,
garden, yard, two entrances, corner, near masjid, investment, negotiable, urgent
near-park wording is stripped first, so "close to the park"
is not read as "has a garden"

A6 Prefer the seller's own answer

where the listing form supplied ac / kitchen, that value overrides the keyword flag

20 binary feature columns
has_ac · has_parking · has_pool · is_corner · near_masjid · ...

B. Grounded LLM extraction — reads content only

src/llm_fill (local Ollama, gemma4:26b)

B1 Normalise the content

digits folded, URLs / phone numbers / emoji to spaces, diacritics removed,
alef and ya unified, whitespace collapsed -> the `clean` column

B2 Build the work queue

a row is queued for a field only when the value is missing, the field applies
to that estate type, and len(clean) >= 10
-> 211,665 rows, each carrying its own need_* flags

B3 Build a per-row JSON Schema

a Pydantic model covering exactly the fields that row needs, with type and
range bounds, passed as Ollama's `format`: decoding is constrained by the
schema, so no reply is ever free-text parsed

B4 Ask the model, once per row

temperature 0, fixed seed, async under a bounded semaphore; retries with
exponential backoff, and a circuit breaker that aborts the whole run
once the failures say the server itself is down

B5 Verify every answer against the text

the value must fall in range AND each quoted evidence span must occur in
`clean` — exact match, else rapidfuzz partial ratio >= 90
-> accepted · accepted_fuzzy · null_not_mentioned · rejected_*

B6 Checkpoint as it goes

append-only part_000000.parquet ... part_000845.parquet; an interrupted run
resumes by skipping the ids already on disk

B7 Gate on measured accuracy

accuracy measured per field x estate type on 400 held-out labelled rows;
below the bar the value is kept but flagged needs_human_review instead of filled

7 extracted attributes
rooms · driver_room · livings · wc · furnished · age · street_width

real_estate_final.parquet — 781,398 rows

keyword flags and gated LLM values written in together; an existing seller-supplied value is never overwritten,
and every field carries a {field}_extraction_status column recording where its value came from

SELL / RENT_REAL_ESTATE_FOR_MODELING.parquet — 533,399 + 145,991 rows

title and content are dropped here: the text has already been turned into columns.
That is why these files are 5.8 MB and 1.5 MB against 250 MB -> notebook/Modeling.ipynb